

Answer the following short questions:

(a) Explain the RIPR model in relation to software testing? Give a small example of how this relates to a test input and a fault in a program (5 points)

(b) Differentiate a fault, error and failure

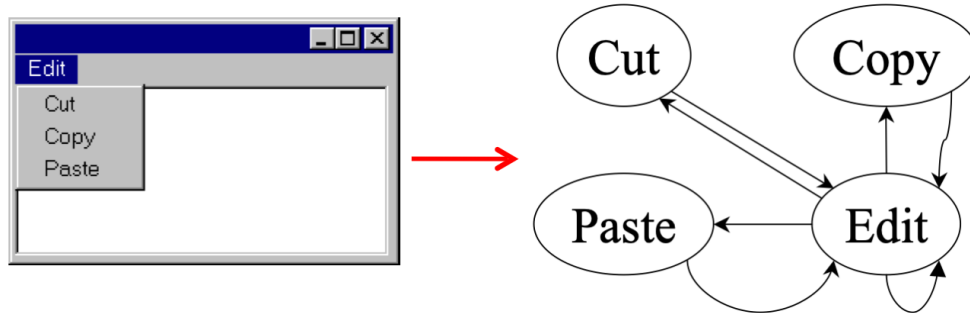
(c) In JUnit we use _____ to designate the test oracle

(d) Explain what a flaky test case is, and ***state 2 ways*** in which it can negatively impact testing.

(e) In the in-class exercise, we used both Guitar and Selenium. One was a model-based testing tool and one was not. Explain (make sure to state why one is a model-based tool)

Question:

Use the following GUI program and event flow graph to answer the questions below. Note that this program has a single menu (expanded) and all actions are shown.



1. How many length 2 test cases would GUITAR generate for this EFG?
2. List a length 3 test case in the EFG
3. Suppose you run the test cases during regression testing. How can you create an (automated) oracle for these tests?

Q4. Suppose you have a Triangle program class called TriangleType.java. This is the same class we used in our class exercise and assignment – you can assume the specifications have not changed. Answer the questions below.

Below is an execution trace from cobertura. The numbers next to the line numbers (second column) state the number of times a line has been covered (e.g. line 18 has been reached 7 times). A line highlight means that the branch (or line) has not been covered completely. For instance, line 10 has never been covered (no class was instantiated) and line 18 has conditionals that are not covered. A color only on the number indicates complete coverage (e.g. line 24 and 25 have been covered completely)

There is a fault (indicated with a comment `//FAULT HERE`). The JUnit tests passed to the application are below the trace.

```

10 0 public class TriangleType
11 {
12     /**
13      * @param s1, s2, s3: sides of the putative triangle
14      * @return enum describing type of triangle
15      */
16     public static Triangle triangle (int s1, int s2, int s3)
17     {
18 7     if (s1 <= 0 || s2 <= 0 || s3 <= 0)
19 2         return (Triangle.INVALID);
20
21 5     if (s1+s2 <= s3 || s2+s3 <= s1 || s1+s3 <= s2)
22 2         return (Triangle.INVALID);
23
24 3     if ((s1 == s2) || (s2 == s3))//FAULT HERE
25 2         return Triangle.EQUILATERAL;
26
27 1     if ((s1 == s2) || (s2 == s3) || (s1 == s3))
28 0         return Triangle.ISOSCELES;
29
30 1     return (Triangle.SCALENE);
31     }
32 }

```

1. Cobertura Output

```

@Parameters public static Collection<Object[]> parameters() {

    List<Object[]> list = new ArrayList<Object[]>();
    list.add(new Object[] {1, 2, 0, Triangle.INVALID});
    list.add(new Object[] {0, 1, 2, Triangle.INVALID});
    list.add(new Object[] {1, 2, 3, Triangle.INVALID});
    list.add(new Object[] {3, 1, 2, Triangle.INVALID});
    list.add(new Object[] {3, 2, 2, Triangle.ISOSCELES});
    list.add(new Object[] {2, 2, 2, Triangle.EQUILATERAL});
    list.add(new Object[] {4, 3, 2, Triangle.SCALENE});
    return list;
}

```

2 JUnit Test Inputs

(a) Do any of the test cases **reach and reveal** the fault? If so, state one that does.
Make sure to confirm by tracing the code.

(c) Find a test case that reaches the fault, should reveal it (based on the specifications), but does not. Explain.

(d) Below is a cobertura code coverage report for these tests. Note that the branch coverage is only 17/22 (it must cover both cases of each condition). Provide another test case that will increase the branch coverage.

Coverage Report - All Packages

Package 	# Classes	Line Coverage		Branch Coverage		Complexity
(default)	2	83%	10/12	77%	17/22	16
All Packages	2	83%	10/12	77%	17/22	16

Classes in this Package 	Line Coverage		Branch Coverage		Complexity
Triangle	100%	2/2	N/A	N/A	0
TriangleType	80%	8/10	77%	17/22	16

3 Cobertura Coverage Report

Q. Is it possible to achieve complete line coverage for the following code? If so, list a set of inputs to achieve this goal. If not, explain and discuss implications.

```
1. if (x<2) {  
2. x=2;  
3. y=x;  
4. }  
5. else {  
6. y=1;  
7. }  
8. if (y<1) {  
9. System.out.println("y is less than 1");  
10. }
```

Q. Short Answers

- (a) Define **regression testing** and give an example of when/how it is used.
- (b) There are two different approaches to generating test suites that cover a model. Given two approaches and provide examples. You can use our class exercises to help with this.
- (c) Differentiate structural and specification based testing.
- (d) What is integration testing (compare against unit and system testing)
- (e) Explain what **continuous integration testing** is and how this is related to test driven development.

Q. Suppose you have the following categories of marbles that can be used in your a program and need to be tested. Each marble has a color and a finish option (shown in parentheses)

- yellow (shiny, striped)
- blue (shiny)
- green (shiny, striped, dots)
- red (dots)
- black (striped)

A marble that is yellow and shiny is written “yellow-shiny” and one that is yellow and striped is written yellow-striped.

Now suppose we design two test criteria, C1 and C2 for this program. The first test criteria (C1) *covers all colors of marbles* and the second criteria (C2) *covers all of the finishes*. Answer the following questions:

(a) Which criterion or criteria does the following test set satisfy (*state all that apply*):

Marbles in test set:

{1 shiny-yellow, 2 striped-yellow, 1 green-dots, 2 red-dots, 1 black-striped}

(b) For the first (or only) criterion you selected, state if it this is *a minimal test set*. Explain and demonstrate why or why not. *If it is not, then provide a minimal test set for this criterion.*

(c) There is a subsumption relationship between C1 and C2. State what the subsumption relationship is and demonstrate with an example.

Q6. Controllability and Observability both impact the *Testability* of a program. Use concrete examples *to explain how both controllability and observability* may be difficult or hard **contrasting two types of systems**, a typical command line program like triangle, and a robotic system that uses sensors to interact with the physical world. **(10 Points)**

controllability

observability